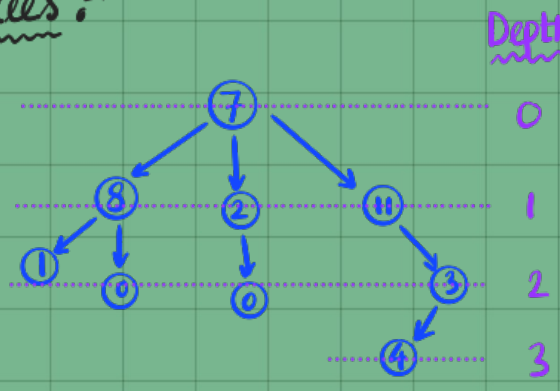
* Trees :-



• Terminologies :-

① Root → Topmost Node

② Parent → Node which connects to the child

③ Child → Node which is connected by another node is its child

④ Leaf → Nodes with no children (external node)

⑤ Internal node → Nodes with atleast one child

⑥ Depth → no of edges from root to the node

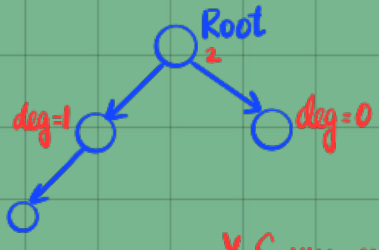
⑦ Height : no of edges from node to the deepest leaf

⑧ Sibling → node belonging to the same parents

⑨ Ancestors & Descendents

* Binary Tree :-

• Binary Tree is a tree which has atmost 2 children for all nodes.



* Summary :-

① Trees are made up of nodes and edges.

② n nodes → (n-1) edges

③ Degree → no of direct children (for a node)

④ Degree of a tree is the highest degree of a node among all the nodes that in the tree.

⑤ Binary Tree → Tree of degree 2
↳ nodes can have 0, 1, 2 children

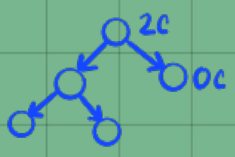
* Types of Binary Trees:

Tree → Non-linear Data Structure
ideal for representing hierarchical data

Array, Linked-list, Stack, Queue } Linear data struct

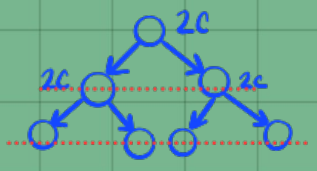
① Full or Strict BT

all nodes have either 0 or 2 children



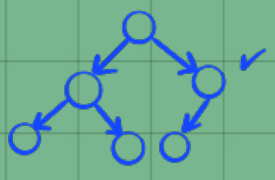
② Perfect BT

internal nodes have 2 children + all leaf nodes are on same level



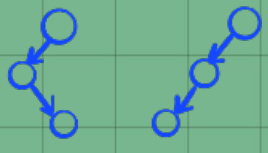
③ Complete BT

all levels are completely filled except possibly the last level + last level must have its keys as left as possible



④ Degenerate Tree

every parent node has exactly one child



⑤ Skewed Trees



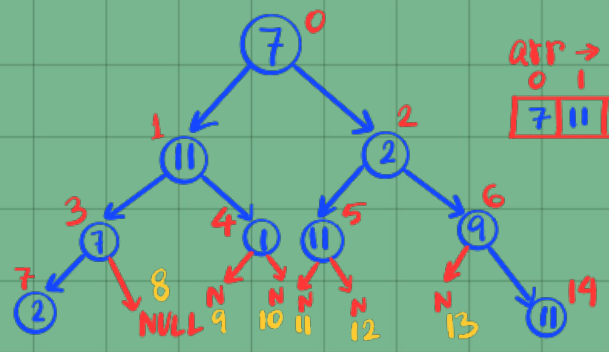
left skewed



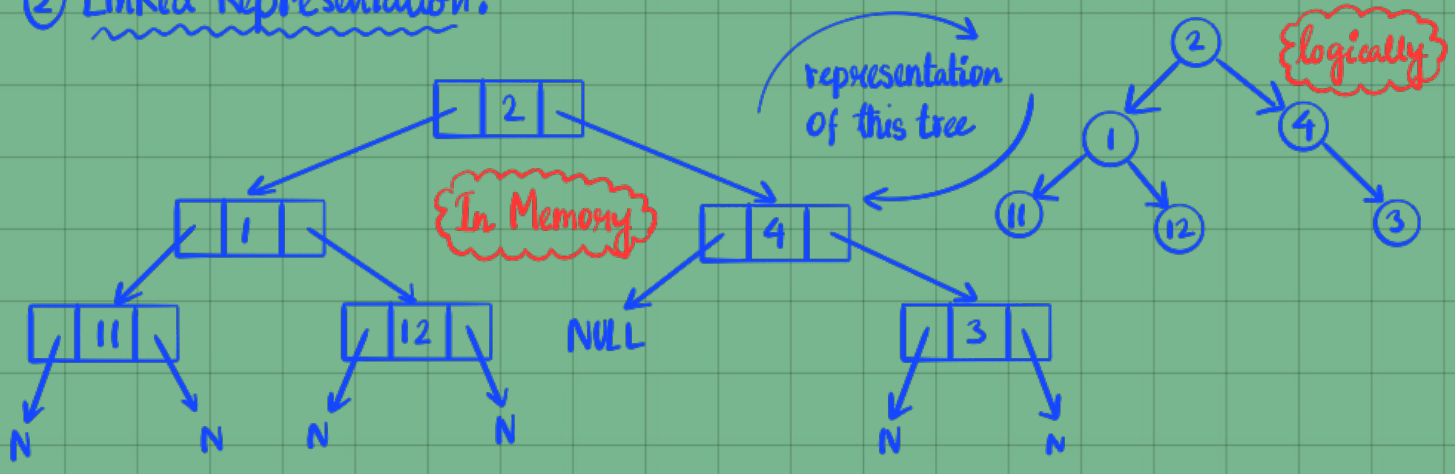
right skewed

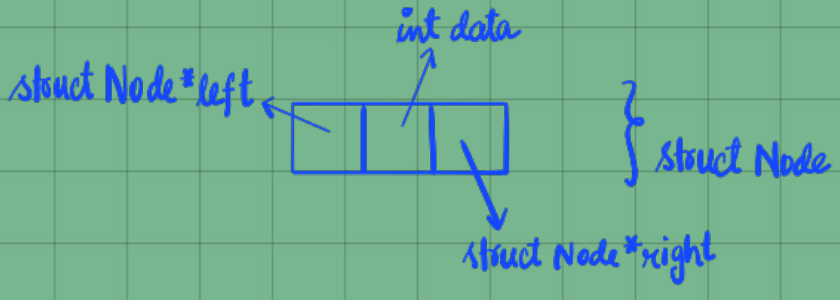
* Representation of BT :-

① Array Representation :



② Linked Representation :-





* Traversal in a Binary Tree :-

* linear data structure : Front to Rear
Rear to Front

① Preorder Traversal

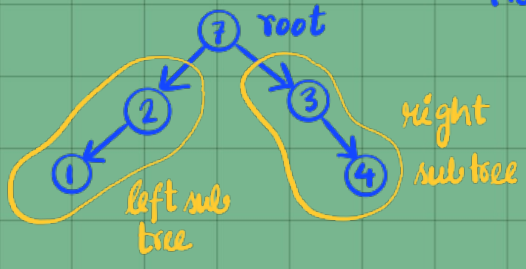
Root → left subtree → right subtree

② Postorder Traversal

left subtree → right subtree → Root

③ Inorder Traversal

left subtree → Root → right subtree

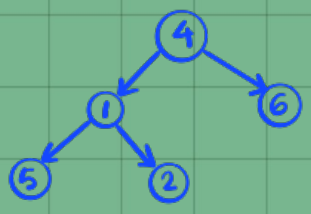


Preorder 7[2[1]][3[4]] → 7 2 1 3 4

Postorder 1 2 4 3 7

Inorder 1 2 7 3 4

* Preorder Traversal :-



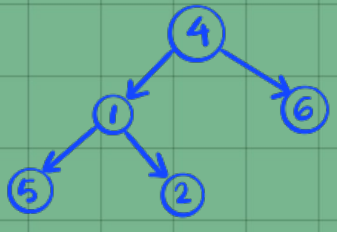
root → left subtree → right subtree

4 → [1 5 2] → 6

4 1 5 2 6

```
void preorder (struct node *root){
    if(root!=NULL){
        printf(root->data);
        preorder (root->left);
        preorder (root->right);
    }
```

* Postorder Traversal :-



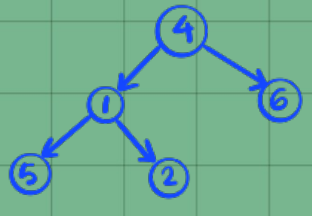
left → right → root

[5 2 1] → 6 → 4

5 2 1 6 4

```
void postorder (node* root)
{   if(root!=NULL){
        postorder(root->left)
        postorder (root->right)
        printf (root->data);
    }
}
```

* Inorder Traversal :-



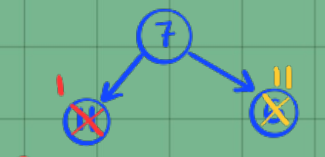
left → root → right

[5 1 2] → 4 → 6

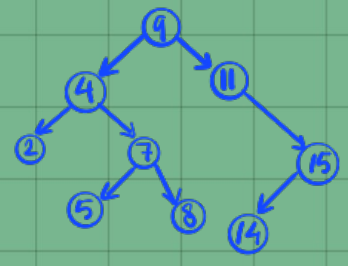
```
void inorder(node*root){
    if(root!=NULL){
        inorder (root->left);
        printf(root->data);
        inorder (root->right);
    }
}
```

* Binary Search Tree :-

(a) it is a type of Binary Tree
(b) all nodes of the left subtree are lesser.
(c) all nodes of the right subtree are greater
(d) left & right subtrees are also BST.
(e) There are no duplicate nodes
(f) Inorder traversal of a BST gives an ascending sorted array.



① not a BST
② not a BST
③ Is a BST



it is a BST

ascending odr
• inorder :- 2 4 5 7 8 9 11 14 15

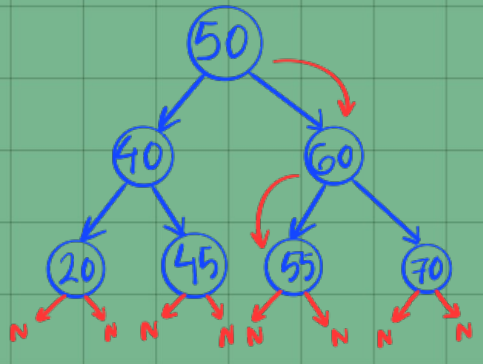
* Searching in BST :-

$T(n) = h \times t \simeq O(\log n)$

$\log n \le h \le n$

in case of skewed tree

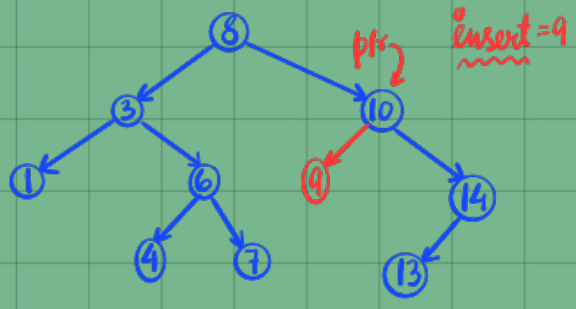
```
struct node* search (Node *root, int key){
  if (root == NULL){
      return NULL;
  }
  if (root -> data == key){
    return root;
  }
  else if (root -> data > key){
      return search(root -> left, key);
  }else {
    return search(root -> right, key);
  }
```



target : 55

* Insertion in a binary search tree :-

(no duplicates allowed)




```
void insert(node* root, int key){
    Node* prev = NULL;
    while(root != NULL){
        prev = root
        if(root->data == key){
            return;
        }
        else if( key < root->data){
                - = root->left;
        }
        else{
            root = root->right;
        }
    }
    // link the key nodes with prev pointer
```

* Deletion of a node in binary search tree :-

case 1: Node is leaf node

case 2: Node is a non-leaf node

case 3: Node is the root node / node in between

Case 1

ⓐ search the node

ⓑ delete the node

Case 2

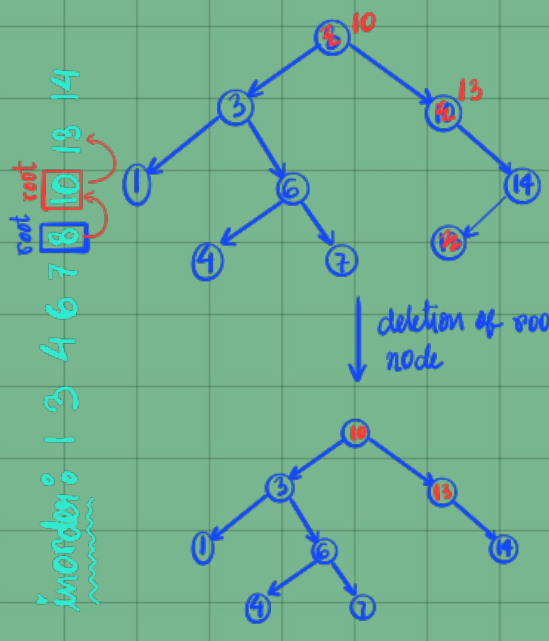
ⓐ node is replaced by inorder predecessor / inorder successor

Case 3

ⓐ search for node

ⓑ search for Inpre / Inpost

ⓒ keep going until the tree has no empty nodes.



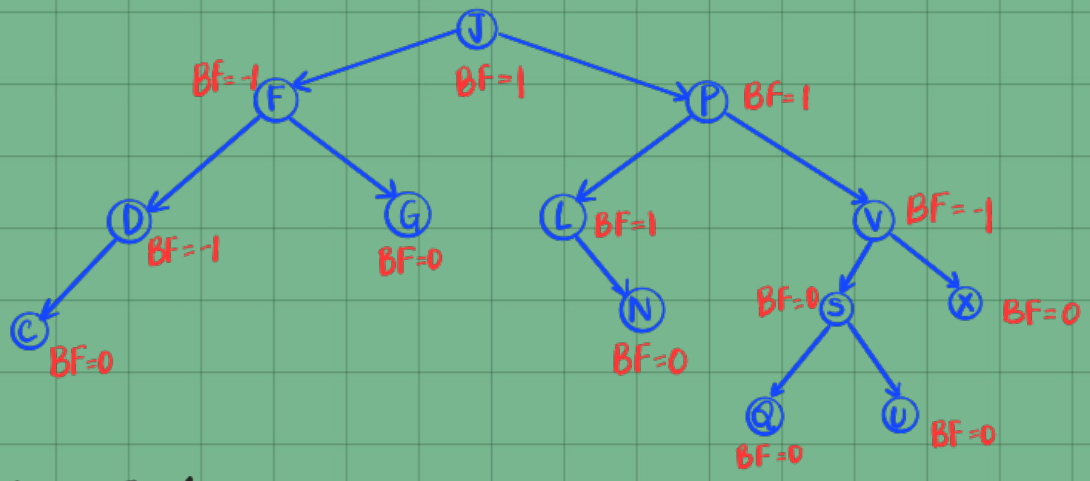
* AVL Tree :-

Need

- almost all operations in binary search tree are of order $O(h)$ where h = height of the tree
- if we don't plan our tree properly, this height can get as high as n, n = no of nodes in BST (skewed tree)
- To guarantee an upper bound of $O(\log n)$ for all these opr, we use balanced tree

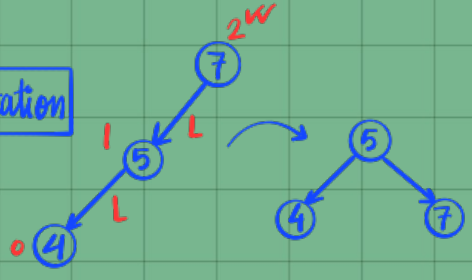
Features

- height balanced BSTs
- height diffn b/w left & right subtrees for every node is ≤ 1
- Balanced Factor = Height of right subtree − height of left subtree $\quad |BF| \leq 1$
- can be $\in [-1, 1]$ for all nodes of an AVL Tree

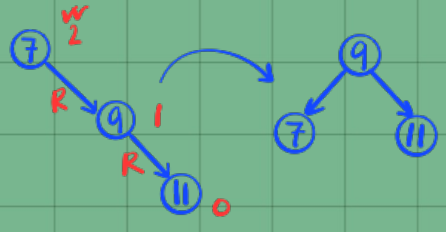


* Rotation in AVL Tree :-

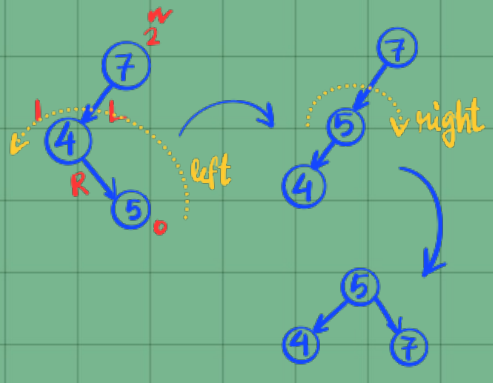
LL rotation



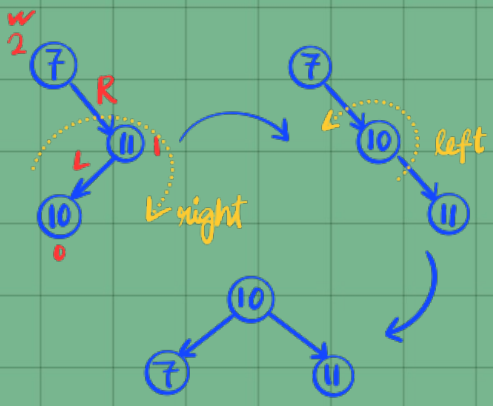
RR rotate



LR rotation



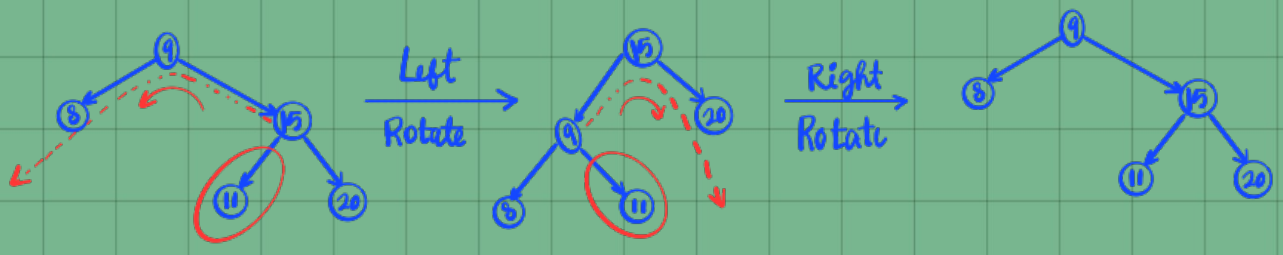
RL rotation



* Rotate Operations :-

ⓐ Left rotate wrt node → Node is moved towards the left

ⓑ Right rotate wrt node → Node is moved towards the right



Balancing AVL Tree after rotation

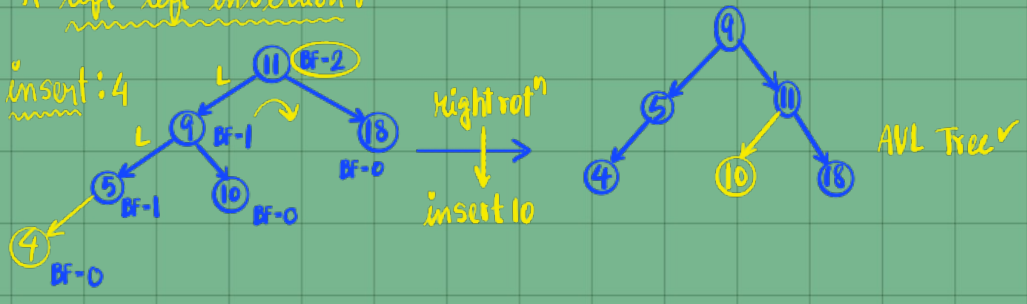
ⓐ left-left rot^n → right rotate once wrt the first imbalanced node

ⓑ right-right rot^n → left rotate once wrt the first imbalanced node.

ⓒ left-right rot^n → left rotate once & then right rotate once.

ⓓ right-left rot^n → right rotate once & left rotate once.

* left-left insertion :-



* Right-left insertion :-

